



Lecture[03]

POINTERS



Pointers

- Powerful, but difficult to master
- Simulate call-by-reference
- Close relationship with arrays and strings



Agenda

- Address and Pointer
- Declaration and Initialization
- Pointer Operation
- Function call by value and call by reference
- Pointer and array



@ of a variable

- Operator &
 - Get the @ of a variable
 - Followed by a variable
- Work with array
 - int a[10];
 - a[0]? a[1]?
 - &a[0]+1=?
 - &a[1]-&a[0]
- How to store an address

```
int n;  
printf("%p\n",&n);
```



Pointer Variable vs. Normal Variable

- Normal variables contain a specific value (direct reference)
- Pointer variables

- Contain memory addresses as their values
- Pointers contain *address* of a variable that has a specific value (indirect reference)
- Indirection - referencing a pointer value

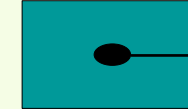
间接引用/解引用：不直接去操作数据，而是通过指针变量里存储的地址，“间接地”找到并操作那个地址上的数据。

count



```
int count=7;  
int* countPtr;  
countPtr=&count;
```

countPtr



count





Declarations and Initialization

- Pointer declarations
 - * used with pointer variables

```
int *myPtr;
```
 - Declares a pointer to an `int` (pointer of type `int *`)
 - Multiple pointers, multiple *

```
int *myPtr1, *myPtr2;  
int *myPtr1, myPtr;
```
 - Can declare pointers to any data type
- Initialize pointers to 0, **NULL**, or an address
 - 0 or **NULL** - points to nothing (**NULL** preferred)



Initialization

- Always initialize your pointer before use what it points to!

```
int *c;  
int k=5;  
*c = 5;
```



Segmentation fault

不初始化就使用指针，会出现段错误：程序试图访问一块不属于它的内存区域，操作系统为了保护内存安全而强制终止了该程序。



Pointer Operators

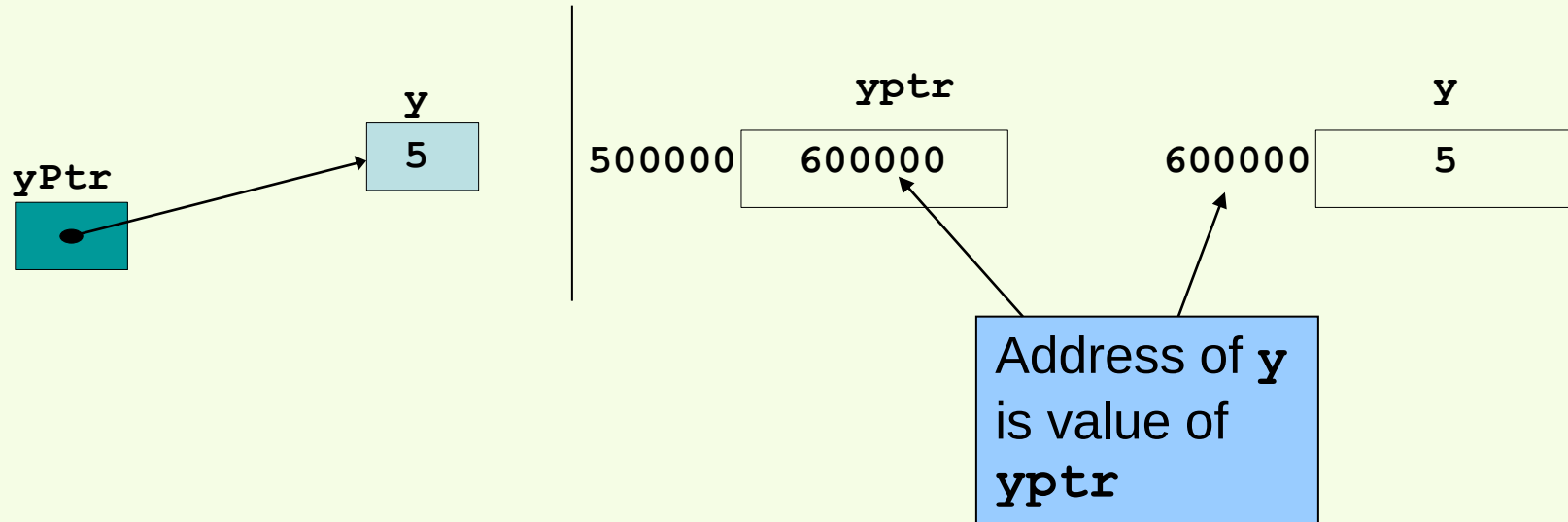
- & (address operator)
 - Returns address of operand

```
int y = 5;
```

```
int *yPtr;
```

```
yPtr = &y; //yPtr gets address of y  
           //yPtr "points to" y
```

Initialization





Pointer Operators

间接引用/解引用操作符：通过指针变量中存储的内存地址，来访问或修改该地址上存储的数据。

- ***** (indirection/dereferencing operator)

- Returns a synonym/alias of what its operand *points to*
返回其操作数所指向内容的一个同义词/别名。

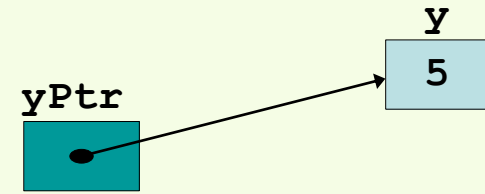
- *yPtr** returns **y** (because **yPtr** points to **y**)

- ***** can be used for assignment

- 可以用于（通过指针）向它所指向的内存地址赋值。

- Returns alias to an object

- *yPtr = 7; // changes y to 7**





Pointer Operators

- ** and & are inverses*
 - They cancel each other out

**&yptr -> * (&yptr) -> * (address of yptr) ->
returns alias of what operand *points* to -> yptr*

*&*yptr -> &(*yptr) -> &(y) -> returns address of
y, which is yptr -> yptr*



```
1  /* Fig. 7.4: fig07_04.c
2     Using the & and * operators */
3  #include <stdio.h>
4
5  int main()
6  {
7      int a;          /* a is an integer */
8      int *aPtr;      /* aPtr is a pointer to an integer */
9
10     a = 7;
11     aPtr = &a;      /* aPtr set to address of a */
12
13     printf( "The address of a is %p"
14            "\nThe value of aPtr is %p", &a, aPtr );
15
16     printf( "\n\nThe value of a is %d"
17            "\nThe value of *aPtr is %d", a, *aPtr );
18
19     printf( "\n\nShowing that * and & are inverses of "
20            "each other.\n&*aPtr = %p"
21            "\n*&aPtr = %p\n", &*aPtr, *&aPtr );
22
23     return 0;
24 }
```

The address of **a** is the value of **aPtr**.

The ***** operator returns an alias to what its operand points to. **aPtr** points to **a**, so ***aPtr** returns **a**.

Notice how ***** and **&** are inverses

The address of a is 0012FF88
The value of aPtr is 0012FF88

The value of a is 7
The value of *aPtr is 7
Showing that * and & are inverses of each other.
&*aPtr = 0012FF88
*&aPtr = 0012FF88



Using the Const Qualifier with Pointers

- **const** qualifier - variable cannot be changed
 - Good idea to have **const** if function does not need to change a variable
 - Attempting to change a **const** is a compiler error
- **const** pointers - point to same memory location
 - Must be initialized when declared

`int *const myPtr = &x;` 指针常量

- Type `int *const` - constant pointer to an `int`

`const int *myPtr = &x;` 常量指针

- Regular pointer to a `const int`, `*myPtr` can not be reassigned with a value

`const int *const Ptr = &x;` 指向常量的指针常量

- `const` pointer to a `const int`



Using the Const Qualifier with Pointers

- 常量指针，指针指向的内容是常量，不可通过此指针修改。但指针本身可以指向别的地址。
 - `const int *p;` // 最常用
 - `int const *p;` // 等价于上一种
 - `const` 修饰的是 `*p`（即指针`p`所指向的内容）。所以 `*p` 是一个常量，不能被修改。但 `p` 本身（即指针变量里存的地址）是可以改变的。
- 指针常量，指针本身是常量，一旦初始化就不能再指向其他地址。但可以通过它修改它所指向地址的内容。
 - `int *const p;`
 - `const` 修饰的是 `p`（即指针变量本身）。所以 `p` 是一个常量，它的值（存储的地址）不能再改变。但 `*p`（它指向的内存内容）是可以修改的。
- 指向常量的指针常量，把两者结合起来，实现指针和指向的内容都不可修改。
 - `const int *const p;`



Using the Const Qualifier with Pointers

类型	写法	指针本身 (p)	指向的内容 (*p)	中文名
常量指针	<code>const int *p</code>	可以改变	不可改变	指向常量的指针
指针常量	<code>int *const p</code>	不可改变	可以改变	指针本身是常量
双重const	<code>const int *const p</code>	不可改变	不可改变	指向常量的指针 常量

- `const` 在 `*` 左边, 定值 (指向的内容不变)
- `const` 在 `*` 右边, 定址 (指针本身不变)



```
1  /* Fig. 7.13: fig07_13.c
2     Attempting to modify a constant pointer to
3     non-constant data */
4
5  #include <stdio.h>
6
7  int main()
8  {
9     int x, y;
10
11     int * const ptr = &x; /* ptr is a constant pointer to an
12                            integer. An integer can be modified
13                            through ptr, but ptr always points
14                            to the same memory location. */
15     *ptr = 7;
16     ptr = &y;
17
18     return 0;
19 }
```

Changing ***ptr** is allowed - **x**
is not a constant.

Changing **ptr** is an error -
ptr is a constant pointer.

FIG07_13.c:

Error E2024 FIG07_13.c 16: Cannot modify a const
object in
function main

*** 1 errors in Compile ***



Calling Functions: Call by Value

- Used when invoking functions
- Call by value
 - Copy of argument passed to function
 - Changes in function do not effect original
 - Use when function does not need to modify argument
 - Avoids accidental changes

```
void addone(int n) {  
    n++;  
}
```

```
int i=10;  
printf("Before: %d\n", i);  
addone(i);  
printf("After: %d\n", i);
```



Calling Functions by Reference

- Call by reference with pointer arguments
 - Pass address of argument using **&** operator
 - Allows you to change actual location in memory
 - Arrays are not passed with **&** because the array name is already a pointer
- ***** operator
 - Used as alias/nickname for variable inside of function

```
void addone(int *n) {  
    (*n)++;  
}
```

***n** used as nickname for the variable passed



```
1  /* Fig. 7.7: fig07_07.c
2     Cube a variable using call-by-reference
3     with a pointer argument */
4
5  #include <stdio.h>
6
7  void cubeByReference( int * );    /* prototype */
8
9  int main()
10 {
11     int number = 5;
12
13     printf( "The original value of number is %d", number );
14     cubeByReference( &number );
15     printf( "\nThe new value of number is %d\n", number );
16
17     return 0;
18 }
19
20 void cubeByReference( int *nPtr )
21 {
22     *nPtr = *nPtr * *nPtr * *nPtr;    /* cube number in main */
23 }
```

Notice how the address of **number** is given - **cubeByReference** expects a pointer (an address of a variable).

Inside **cubeByReference**, ***nPtr** is used (***nPtr** is **number**).

The original value of number is 5
The new value of number is 125



Swap function Using Call-by-reference

- Implement swap function
 - Swap two elements
 - **swap** function must receive address (using **&**) of array elements
 - Array elements have call-by-value default
 - Using pointers and the ***** operator, **swap** can switch array elements

```
void swap( int *element1Ptr, int *element2Ptr )  
{  
    int hold = *element1Ptr;  
    *element1Ptr = *element2Ptr;  
    *element2Ptr = hold;  
}
```



The Relationship Between Pointers and Arrays

- Arrays and pointers closely related
 - Array name like a **constant** pointer
 - Pointers can do array subscripting operations
- Declare an array **b[5]** and a pointer **bPtr**

```
int *bPtr = b;
```

Array name actually a address of first element

OR

```
int *bPtr = &b[0]
```

Explicitly assign **bPtr** to address of first element

```
int a[10]={0};  
int b[] = a;
```



```
int a[10]={0};  
int *p=a;
```





The Relationship Between Pointers and Arrays

- Element `b[n]`

- can be accessed by `*( bPtr + n )`

- `n` - offset (pointer/offset notation)

- Array itself can use pointer arithmetic.

- `b[3]` same as `*(b + 3)`

- Pointers can be subscripted (pointer/subscript notation)

- `bPtr[3]` same as `b[3]`



Swap function on an array

- Implement swap function on an array
 - Swap two elements in an array
 - The location of the elements are given

```
void swap( int *element1Ptr, int *element2Ptr )  
{  
    int hold = *element1Ptr;  
    *element1Ptr = *element2Ptr;  
    *element2Ptr = hold;  
}
```

Swap two elements

```
void swap(int *arr, int len, int i, int j){  
    if (i<len)&&(j<len){  
        int hold =*(arr+i);  
        *(arr+i) = *(arr+j);  
        *(arr+j) = hold;  
    }  
}
```

```
void swap(int *arr, int len, int i, int j){  
    if (i<len)&&(j<len){  
        int hold =arr[i];  
        arr[i] = arr[j];  
        arr[j] = hold;  
    }  
}
```

Equivalent



Summary

- Address and Pointer
- Declaration and Initialization
- Pointer Operation
- Function call by value and call by reference
- Pointer and array



Agenda

- Pointer Expressions and Pointer Arithmetic
- Arrays of Pointers



Pointer Expressions and Pointer Arithmetic

- Arithmetic operations can be performed on pointers
 - *, & has higher precedence than arithmetic
 - Increment/decrement pointer (++ or --)
 - Add an integer to a pointer(+ or += , - or -=)
 - Pointers may be subtracted from each other
 - Operations meaningless unless performed on an array

*p+1
*(p+1)



Pointer Expressions and Pointer Arithmetic

- 5 element `int` array on machine with 4 byte `ints`

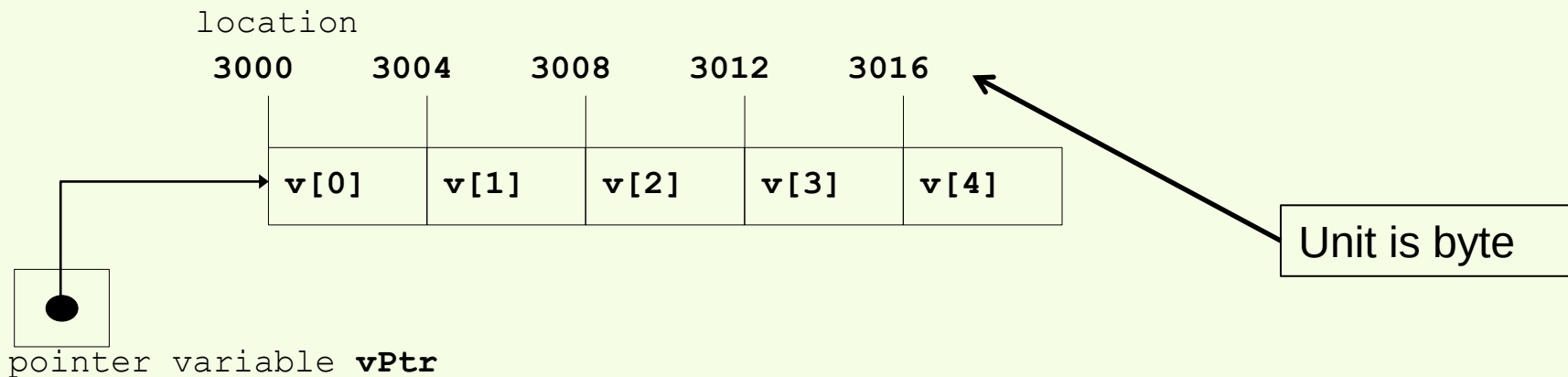
- `vPtr` points to first element `v[0]`

- at location 3000. (`vPtr = 3000`)

- `vPtr += 2`; sets `vPtr` to 3008

- `vPtr` points to `v[2]` (incremented by 2), but machine has 4 byte `ints`.

虽然在代码里让指针的值增加了2 (`vPtr += 2`)，但它在内存中的实际地址值并不是简单地增加2，而是增加了 $2 * \text{sizeof}(\text{int})$ ，也就是 $2 * 4 = 8$ 个字节。这是因为每个 `int` 元素在内存中占用了4个字节的空间。





Pointer Expressions and Pointer Arithmetic

- Subtracting pointers

- Returns number of elements from one to the other.

```
vPtr2 = &v[2];
```

```
vPtr = &v[0];
```

```
vPtr2 - vPtr == 2.
```

- Pointer comparison (<, == , >)

- See which pointer points to the higher numbered array element
- Also, see if a pointer points to 0



Pointer Expressions and Pointer Arithmetic

- Pointers of the same type can be assigned to each other
 - If not the same type, a cast operator must be used
 - Exception: pointer to **void** (type **void ***)
 - Generic pointer, represents any type
 - No casting needed to convert a pointer to **void** pointer
 - **void** pointers cannot be dereferenced
不能直接使用 * 操作符来获取或修改 void* 指针所指向的数据。
- Function malloc return a void pointer

```
void *malloc(size_t size)
```

```
int* p = (int *) malloc ( sizeof(int) * 100 )
```

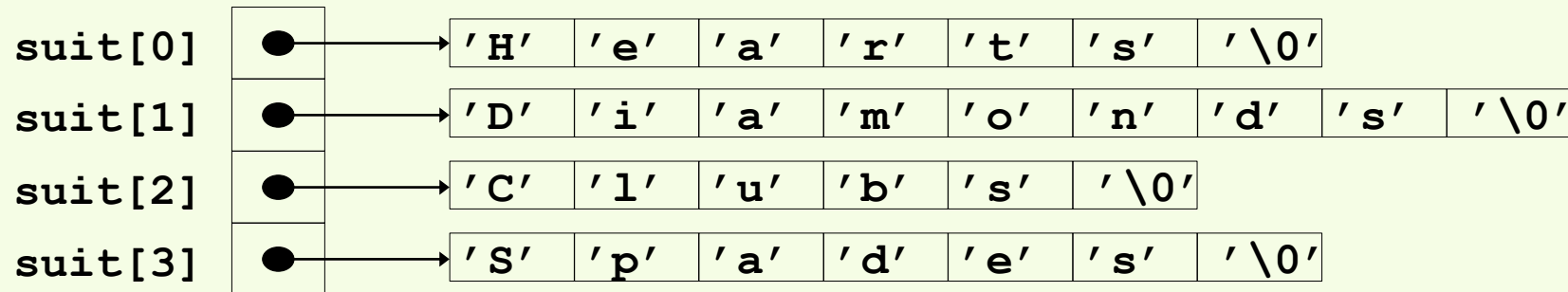


Arrays of Pointers

- Arrays can contain pointers - array of strings

```
char *suit[4] = {"Hearts", "Diamonds", "Clubs", "Spades"};
```

- String: pointer to first character
- char *** - each element of **suit** is a pointer to a **char**
- Strings not actually in array - only *pointers* to string in array



- suit** array has a fixed size, but strings can be of any size.

suit数组的大小是固定的，但字符串的大小可以是任意的。



Case Study: A Card Shuffling and Dealing Simulation

- Card shuffling program
 - Use array of pointers to strings
 - Use double scripted array (suit, face)

		Ace	Two	Three	Four	Five	Six	Seven	Eight	Nine	Ten	Jack	Queen	King
	0	1	2	3	4	5	6	7	8	9	10	11	12	
Hearts	0													
Diamonds	1													
Clubs	2													
Spades	3													

`deck[2][12]` represents the King of Clubs

Clubs King

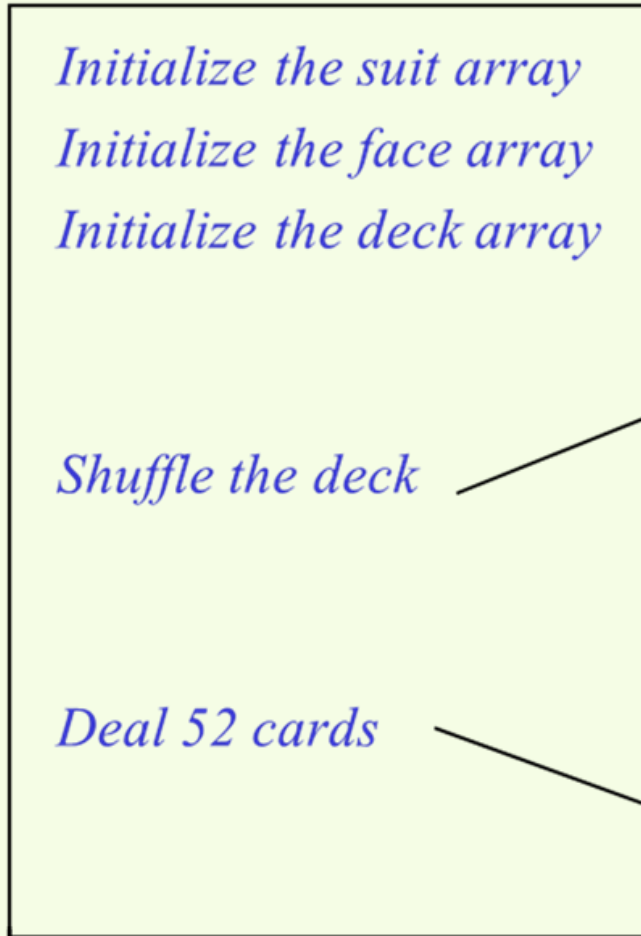
- The numbers 1-52 go into the array - this is the order they are dealt



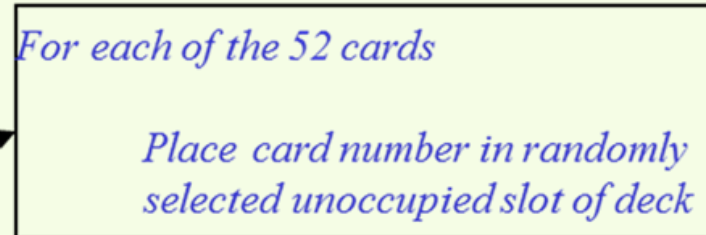
Case Study: A Card Shuffling and Dealing Simulation

- Pseudocode - Top level: *Shuffle and deal 52 cards*

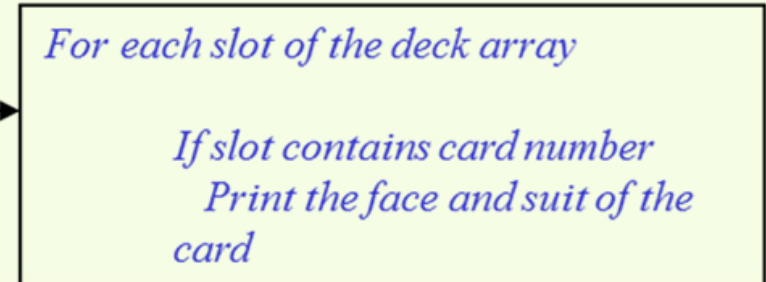
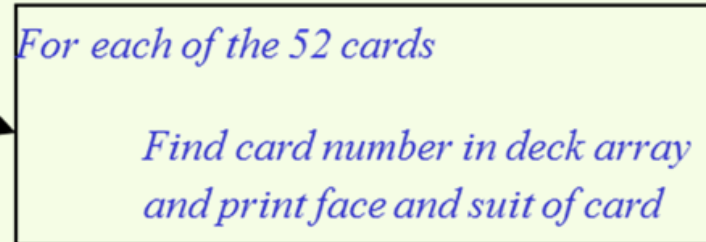
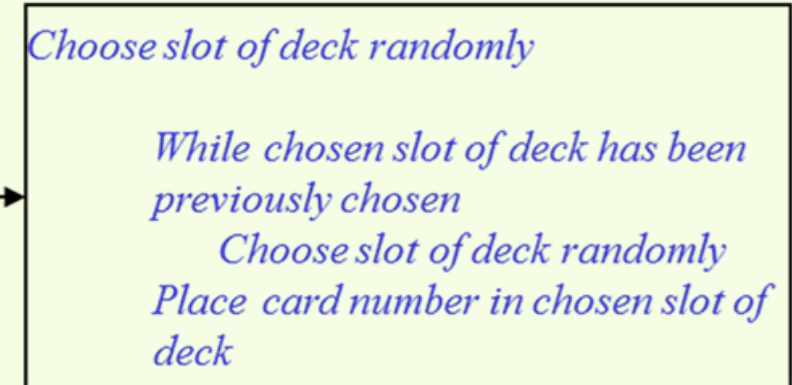
First refinement



Second refinement



Third refinement





```
1  /* Fig. 7.24: fig07_24.c
2      Card shuffling dealing program */
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <time.h>
6
7  void shuffle( int [][] [ 13 ] );
8  void deal( const int [][] [ 13 ], const char *[], const char *[] );
9
10 int main()
11 {
12     const char *suit[ 4 ] =
13         { "Hearts", "Diamonds", "Clubs", "Spades" };
14     const char *face[ 13 ] =
15         { "Ace", "Deuce", "Three", "Four",
16           "Five", "Six", "Seven", "Eight",
17           "Nine", "Ten", "Jack", "Queen", "King" };
18     int deck[ 4 ][ 13 ] = { 0 };
19
20     srand( time( 0 ) );
21
22     shuffle( deck );
23     deal( deck, face, suit );
24
25     return 0;
26 }
27
```



shuffle

```
28 void shuffle( int wDeck[][ 13 ] )
29 {
30     int row, column, card;
31
32     for ( card = 1; card <= 52; card++ ) {
33         do {
34             row = rand() % 4;
35             column = rand() % 13;
36         } while( wDeck[ row ][ column ] != 0 );
37
38         wDeck[ row ][ column ] = card;
39     }
40 }
41
```

Q1: shuffle 函数的参数 `int wDeck[][13]` 是什么意思？

A1: `int wDeck[][13]` 是一个二维数组的形参，其中第一维的长度未知，第二维的长度为13。即，这个数组的每一行有13个元素，但是行数未知。当定义二维数组作为函数参数时，行数可以省略，列数必须指定，这样编译器才能正确计算数组元素的内存布局。

`wDeck` 的类型是一个指向包含 13 个 `int` 类型元素的数组的指针，即 `int (*)[13]`。

Q2: `int wDeck[][13]` 还可以写成什么形式？

A2: 还可以写作 `int (*wDeck)[13]`。这两种形式是等价的。`(*wDeck)`代表`wDeck`是一个指针，`int[13]`代表`wDeck`指向的是一个长度为13的`int`数组。这里，`int (*wDeck)[13]`代表`wDeck`是一个指向`int[13]`类型的指针。



shuffle

```
28 void shuffle( int wDeck[][ 13 ] )
29 {
30     int row, column, card;
31
32     for ( card = 1; card <= 52; card++ ) {
33         do {
34             row = rand() % 4;
35             column = rand() % 13;
36         } while( wDeck[ row ][ column ] != 0 );
37
38         wDeck[ row ][ column ] = card;
39     }
40 }
41
```

Q3: 为什么不直接使用int wDeck[4][13] 作为参数?

A3: 函数shuffle被设计为可以处理任何4行、13列的二维数组，而不仅仅是某个名为 wDeck 的特定数组。使用 int wDeck[][13] 作为参数形式，使得这个函数成为一个通用的洗牌工具。

Q4: shuffle 函数洗牌的原理是什么?

A4: 随机将每张牌插入牌堆的一个位置。



deal

```
42 void deal( const int wDeck[][ 13 ], const char *wFace[],
43           const char *wSuit[] )
44 {
45     int card, row, column;
46
47     for ( card = 1; card <= 52; card++ )
48
49         for ( row = 0; row <= 3; row++ )
50
51             for ( column = 0; column <= 12; column++ )
52
53                 if ( wDeck[ row ][ column ] == card )
54                     printf( "%5s of %-8s%c",
55                             wFace[ column ], wSuit[ row ],
56                             card % 2 == 0 ? '\n' : '\t' );
57 }
```

Q5: deal 函数的参数 `const int wDeck[][13]`, `const char *wFace[]`, `const char *wSuit[]` 中出现了三个 `const`。这三个 `const` 的作用是什么？

A5: 这三个 `const` 分别修饰了 `wDeck`, `wFace`, `wSuit`，表示这三个指针指向的值对函数来说是只读的，函数内部不能修改这三个指针指向的值的值。
`wDeck` 的类型是 `const int (*)[13]`，表示 `wDeck` 是一个指向 `const int[13]` 类型的指针。
`wFace` 和 `wSuit` 的类型都是 `const char *`，是指向 `const char *` 类型的指针。



Pointers to Functions

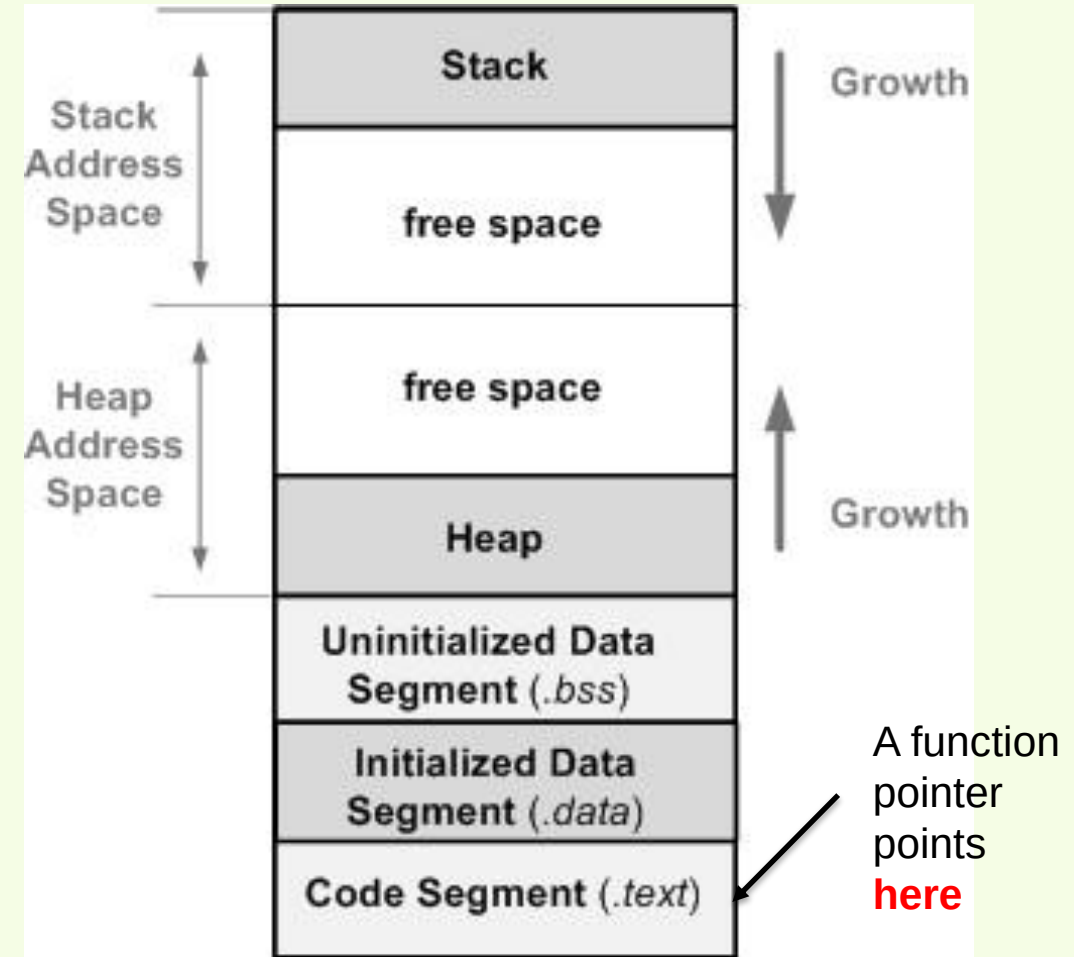
- Pointer to function
 - Contains address of function
 - Similar to how array name is address of first element
 - Function name is starting address of code that defines function
- Function pointers can be
 - Passed to functions
 - Stored in arrays
 - Assigned to other function pointers

```
int (*p)(int, int);  
int c=(*p)(a,b);
```



Memory Layout of a C program

- **Text:** code segment contains the executable instructions of the program. It is usually read-only and shared among multiple instances of the program.
- **Data:** stores global and static variables that are initialized by the programmer. (static int i = 10 or global int i=10).
- **BSS:** stores **uninitialized** global and static variables
- **Heap:** deal with dynamic memory allocation and is managed by malloc, realloc and free.
- **Stack:** stores automatic variable, use a stack/LIFO structure





内存布局图

- 代码段存放程序的执行代码（机器指令）。由编译器生成。
 - 通常是只读的，防止程序意外修改其指令。
 - 可以被多个运行的相同程序共享，以节省内存。
- 数据段存放已初始化的全局变量和静态变量（包括静态全局变量和静态局部变量）。
 - 在程序开始前就分配好空间并赋初值。
- BSS 段 (Block Started by Symbol)存放未初始化的全局变量和静态变量。
 - 在程序开始执行前，内核会将此段中的数据初始化为0（或空指针）。
 - BSS段不占用可执行文件的实际磁盘空间，它只在运行时在内存中分配。



内存布局图

- 堆存放动态分配的内存。
 - 向上增长（向高地址方向）。
 - 内存的分配和释放需要程序员手动管理，否则会导致内存泄漏。
 - 分配速度相对较慢。
- 栈存放局部变量（函数内部非static的变量）、函数参数、返回地址（函数调用结束后应该回到哪里继续执行）、保存的上下文信息。
 - 向下增长（向低地址方向）。
 - 由编译器自动管理分配和释放（函数调用时分配，函数返回时释放）。
 - 分配速度非常快。
 - 空间通常有限，如果递归过深或局部变量过大，会导致栈溢出（Stack Overflow）。



```
1  /* Fig. 7.26: fig07_26.c
2      Multipurpose sorting program using function pointers */
3  #include <stdio.h>
4  #define SIZE 10
5  void bubble( int [], const int, int (*)( int, int ) );
6  int ascending( int, int );
7  int descending( int, int );
8
9  int main()
10 {
11
12     int order,
13         counter,
14         a[ SIZE ] = { 2, 6, 4, 8, 10, 12, 89, 68, 45, 37 };
15
16     printf( "Enter 1 to sort in ascending order,\n"
17            "Enter 2 to sort in descending order: " );
18     scanf( "%d", &order );
19     printf( "\nData items in original order\n" );
20
21     for ( counter = 0; counter < SIZE; counter++ )
22         printf( "%5d", a[ counter ] );
23
24     if ( order == 1 ) {
25         bubble( a, SIZE, ascending );
26         printf( "\nData items in ascending order\n" );
27     }
28     else {
29         bubble( a, SIZE, descending );
30         printf( "\nData items in descending order\n" );
31     }
32 }
```

Notice the function
pointer parameter.

- 1. Initialize array.
- 2. Prompt for ascending or descending sorting.
 - 2.1 Put appropriate function pointer into bubble.
 - 2.2 Call bubble.
- 3. Print results.



```
33     for ( counter = 0; counter < SIZE; counter++ )
34         printf( "%5d", a[ counter ] );
35
36     printf( "\n" );
37
38     return 0;
39 }
40
41 void bubble( int work[], const int size,
42             int (*compare)( int, int ) )
43 {
44     int pass, count;
45
46     void swap( int *, int * );
47
48     for ( pass = 1; pass < size; pass++ )
49
50         for ( count = 0; count < size - 1; count++ )
51
52             if ( (*compare)( work[ count ], work[ count + 1 ] ) )
53                 swap( &work[ count ], &work[ count + 1 ] );
54 }
55
56 void swap( int *element1Ptr, int *element2Ptr )
57 {
58     int temp;
59
60     temp = *element1Ptr;
61     *element1Ptr = *element2Ptr;
62     *element2Ptr = temp;
63 }
64
```

ascending and
descending return true or
false. bubble calls swap if
the function call returns true.

Notice how function pointers
are called using the
dereferencing operator.
The * is not required, but
emphasizes that **compare**
is a function pointer and not
a function.



```
65int ascending( int a, int b )
66{
67    return b < a;    /* swap if b is less than a
68}
69
70int descending( int a, int b )
71{
72    return b > a;    /* swap if b is greater
73}
```

```
Enter 1 to sort in ascending order,
Enter 2 to sort in descending order: 1
```

```
Data items in original order
```

```
2   6   4   8  10  12  89  68  45  37
```

```
Data items in ascending order
```

```
2   4   6   8  10  12  37  45  68  89
```

```
Enter 1 to sort in ascending order,
Enter 2 to sort in descending order: 2
```

```
Data items in original order
```

```
2   6   4   8  10  12  89  68  45  37
```

```
Data items in descending order
```

```
89  68  45  37  12  10   8   6   4   2
```



Summary

- Pointer with array and arithmetic ++, --
 - Very careful with multiple-script
 - Very careful with string and character array
- Const protection related to usage of function
- Pointer to function
 - Function as data and have an @